

Ejercicios evaluables UA3

Esta actividad aborda los criterios de evaluación del RA.

Instrucciones

● Todas las actividades que se realicen deben entregarse mediante un enlace al repositorio de Github y un documento (si se pide alguna evidencia o se plantean preguntas)

DAW_ OPT_UA3_ApellidosNombre.pdf

- Las entregas deberán realizarse en la tarea correspondiente en el formato especificado antes de la hora de finalización. Cualquier tarea que no cumpla estos requisitos tendrá una calificación de 0.
- Se pedirá que defiendas tu práctica. Si no puedes justificar tu trabajo o responder las preguntas que se te plantean, la calificación de la tarea será de 0.

Sobre el uso de IA y fuentes externas

Puedes utilizar herramientas como buscadores o inteligencia artificial como apoyo, pero no como sustituto de tu trabajo. Las respuestas deben estar redactadas por ti, con tus propias ideas y razonamientos.

Se valorará especialmente el análisis personal y la justificación razonada.

Para Python/Django

Ejercicio 1

En DWES Semana 2 se ha implementado el registro de usuarios mediante **POST** a **/api/auth/register/**.

Crea tests que comprueben casos válidos y no válidos.

- ☐ Caso válido (201) y respuesta con id y username (sin contraseña).
- ☐ Casos no válidos (400 validation_error): JSON vacío, faltan campos, contraseña corta, username repetido.

Ejercicio 2

En DWES Semana 2 se ha implementado el inicio de sesión mediante **POST** a **/api/auth/login/**.

Crea tests que comprueben casos válidos y no válidos.

- ☐ Caso válido (200).
- ☐ Casos no válidos: credenciales incorrectas (401 unauthorized, mensaje "Credenciales incorrectas"), y validación (400 validation_error).

Ejercicio 3

En DWES Semana 2 se ha implementado la comprobación de sesión mediante **GET** a **/api/users/me/**.

Crea tests que comprueben casos válidos y no válidos.

- ☐ Sin autenticar: 401 unauthorized, mensaje "No autenticado".
- ☐ Tras login correcto: 200 que devuelve el id y username.

Ejercicio 4

En la semana anterior ya se crearon tests para el listado mediante **GET** a **/api/library/entries/**.

Modifica esos tests para adaptarlos a la autenticación y al uso de usuarios.

- ☐ Sin autenticar: 401 unauthorized, mensaje "No autenticado".
- ☐ Autenticado: 200.
- ☐ Dos usuarios: cada uno ve solo sus entradas.

Ejercicio 5

En la semana anterior ya se crearon tests para el detalle mediante **GET** a **/api/library/entries/{id}/**.

Modifica esos tests para adaptarlos a la autenticación y a la ocultación de recursos de otros usuarios.

- ☐ Sin autenticar: 401 unauthorized, mensaje "No autenticado".
- ☐ Autenticado y entrada propia: 200.
- ☐ Autenticado y entrada de otra persona: 404 not_found, mensaje "La entrada solicitada no existe".

Ejercicio 6

En la semana anterior ya se crearon tests para la creación mediante **POST** a **/api/library/entries/**.

Modifica esos tests para adaptarlos a la autenticación y al hecho de que la entrada queda asociada al usuario autenticado.

- ☐ Sin autenticar: 401 unauthorized, mensaje "No autenticado".
- ☐ Autenticado: 201.
- ☐ Aislamiento: lo creado por un usuario no aparece en el listado del otro.

Ejercicio 7

Ejecuta los tests con *coverage* y analiza el estado real de protección del proyecto. Recuerda que tienes los comandos necesarios para ejecutarlo en el README del proyecto:

- ☐ Indica el porcentaje global de cobertura.
- ☐ Identifica los módulos/archivos con menor cobertura.
- ☐ Señala qué partes (funciones/métodos/bloques) aparecen como no cubiertas.
- ☐ Propón qué dos zonas priorizarías para aumentar cobertura y con qué tipo de tests (unitarios o vistas).

Para Java

Endpoints a comprobar

1. GET /api/health/
2. POST /api/tasks/
3. GET /api/tasks/
4. GET /api/stats/summary

Ejercicio 1 (Resuelto)

Crea el archivo: src/test/java/HealthEndpointTest.java

Copia este código y ejecútalo:

```
import org.junit.jupiter.api.Test;

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

import static org.junit.jupiter.api.Assertions.*;

public class HealthEndpointTest {
    @Test
    void health_devuelve200_y_status_ok() throws Exception {
        HttpClient client = HttpClient.newHttpClient();

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("http://localhost:8080/api/health/"))
            .GET()
            .build();

        HttpResponse<String> response = client.send(request,
            HttpResponse.BodyHandlers.ofString());

        assertEquals(200, response.statusCode());
        assertTrue(response.body().contains("\"status\": \"ok\""));
    }
}
```

Ejercicio 2

Crea el archivo: `src/test/java/CreateTaskEndpointTest.java`

El test debe llamarse exactamente:

- `post_tasks_devuelve201_y_campos_basicos()`

Qué debe comprobar:

- Hace POST `http://localhost:8080/api/tasks/`
- Envía JSON con:
 - `title: "Comprar pan"`
 - `priority: 3`
 - `tags: ["casa"]`
- La respuesta tiene:
 - `statusCode == 201`
 - El body contiene:
 - `"title": "Comprar pan"`
 - `"priority": 3`
 - `"done": false`
 - `"id":` (con algún número)

Ejercicio 3

Crea el archivo: `src/test/java/ListTasksEndpointTest.java`

El test debe llamarse exactamente:

- `get_tasks_devuelve200_y_lista()`

Qué debe comprobar:

- Hace GET `http://localhost:8080/api/tasks/`
- La respuesta tiene:
 - `statusCode == 200`
 - El body contiene la clave `"tasks"`

Ejercicio 4

Crea el archivo: `src/test/java/StatsEndpointTest.java`

El test debe llamarse exactamente:

- `get_stats_summary_devuelve200_y_claves()`

Qué debe comprobar:

- Hace GET `http://localhost:8080/api/stats/summary`
- La respuesta tiene:
 - `statusCode == 200`
 - El body contiene estas claves:
 - `"total"`
 - `"done"`
 - `"done_ratio"`
 - `"top_priority_titles"`

Rúbrica de evaluación de la tarea

Criterio de evaluación	Nivel alto (9-10)	Nivel medio (5-9)	Nivel Bajo (1-5)	No logrado (0)